```
# ============================================================
# BIS MARKGUARD - FAKE / RISKY BIS PRODUCT PREDICTION SYSTEM
# ============================================================
#
# Main Model:
#   Random Forest Classifier
#
# Features:
#   - Data cleaning
#   - Feature engineering
#   - Model comparison
#   - Random Forest hyperparameter tuning
#   - Cross-validation
#   - Accuracy / Precision / Recall / F1
#   - Confusion Matrix
#   - Feature importance
#   - New product prediction
#   - Confidence score
#   - Basic BIS warning checks
#
# NOTE:
# The supplied dataset is synthetic. High accuracy on synthetic
# data does NOT prove real-world counterfeit detection accuracy.
# ============================================================
```

```
import os
import re
import warnings
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import (
    train_test_split,
    StratifiedKFold,
    RandomizedSearchCV,
    cross_val_score
)

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.ensemble import (
    RandomForestClassifier,
    ExtraTreesClassifier
```

```
)

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

warnings.filterwarnings("ignore")

# ============================================================
# 1. SETTINGS
# ============================================================

RANDOM_STATE = 42

CSV_FILE = "bis_markguard_cleaned_synthetic_dataset_1000.csv"
```

```
MODEL_FILE = "bis_markguard_random_forest_model.joblib"

# ============================================================
# 2. LOAD DATASET
# ============================================================

def load_dataset():

    if not os.path.exists(CSV_FILE):

        print("\nERROR: CSV file not found!")
        print("Make sure the CSV file is in the same folder as this Python file.")
        print("\nExpected filename:")
        print(CSV_FILE)

        return None

    data = pd.read_csv(CSV_FILE)

    print("\n================================================")
    print("DATASET LOADED SUCCESSFULLY")
    print("================================================")

    print("Rows    :", data.shape[0])
```

```
    print("Columns :", data.shape[1])


    print("\nColumn names:")
    for column in data.columns:
        print("-", column)


    return data


# ================================================================
# 3. DATASET INFORMATION
# ================================================================


def dataset_information(data):


    print("\n================================================")
    print("DATASET INFORMATION")
    print("================================================")


    print("\nFirst 5 records:")
    print(data.head())


    print("\nMissing values:")
    print(data.isnull().sum())
```

```
    print("\nDuplicate rows:", data.duplicated().sum())

    if "risk_label" in data.columns:

        print("\nTarget distribution:")
        print(data["risk_label"].value_counts())

# ============================================================
# 4. FEATURE ENGINEERING
# ============================================================

def build_features(data):

    x = data.copy()

    text_columns = [
        "product_name",
        "product_category",
        "observed_claim",
        "candidate_license_number",
        "candidate_is_number",
        "claimed_license_holder"
    ]
```

```
# ------------------------------------------------------------
# Convert text columns to strings
# ------------------------------------------------------------

for column in text_columns:

    if column in x.columns:

        x[column] = (
            x[column]
            .fillna("")
            .astype(str)
            .str.strip()
        )

# ------------------------------------------------------------
# Licence information
# ------------------------------------------------------------

if "candidate_license_number" in x.columns:

    x["has_license"] = (
        x["candidate_license_number"] != ""
    ).astype(int)
```

```
    x["license_length"] = (
        x["candidate_license_number"].str.len()
    )

    x["license_has_digit"] = (
        x["candidate_license_number"]
        .str.contains(r"\d", regex=True)
        .astype(int)
    )

    x["license_has_demo"] = (
        x["candidate_license_number"]
        .str.contains(
            "DEMO",
            case=False,
            regex=False
        )
        .astype(int)
    )

# ------------------------------------------------------------
# Manufacturer information
# ------------------------------------------------------------
```

```
if "claimed_license_holder" in x.columns:

    x["has_holder"] = (
        x["claimed_license_holder"] != ""
    ).astype(int)

    x["holder_length"] = (
        x["claimed_license_holder"].str.len()
    )

# ------------------------------------------------------------
# IS Number information
# ------------------------------------------------------------

if "candidate_is_number" in x.columns:

    x["has_is_number"] = (
        x["candidate_is_number"] != ""
    ).astype(int)

    x["standard_length"] = (
        x["candidate_is_number"].str.len()
    )
```

```
    x["standard_known_format"] = (
        x["candidate_is_number"]
        .str.match(
            r"^IS\s+\d+",
            na=False
        )
        .astype(int)
    )

# ------------------------------------------------------------
# Certification claim
# ------------------------------------------------------------

if "observed_claim" in x.columns:

    claim = x["observed_claim"].str.lower()

    x["claim_strength"] = 0

    x.loc[
        claim.str.contains(
            "certified|standard mark|isi|bis",
            regex=True
```

```
            ),
            "claim_strength"
        ] = 2

        x.loc[
            claim.str.contains(
                "visible",
                regex=True
            ),
            "claim_strength"
        ] = 3

    # Convert blank strings to missing values
    x = x.replace(
        r"^\s*$",
        np.nan,
        regex=True
    )

    return x

# ============================================================
# 5. PREPARE TRAINING DATA
# ============================================================
```

```
def prepare_training_data(data):

    TARGET = "risk_label"

    if TARGET not in data.columns:

        raise ValueError(
            "The dataset does not contain the required "
            "'risk_label' column."
        )

    # ------------------------------------------------------------
    # Remove target leakage and metadata
    # ------------------------------------------------------------

    leakage_columns = [
        "risk_label",
        "case_id",

        # Synthetic-data metadata
        "dataset_status",
        "demo_record_status",
        "risk_reason",
```

```
        "source_type",
        "source_url",
        "record_date",
        "is_synthetic",
        "cleaning_status"
    ]

    x = build_features(data)

    x = x.drop(
        columns=[
            column
            for column in leakage_columns
            if column in x.columns
        ],
        errors="ignore"
    )

    y = data[TARGET].astype(str)

    return x, y

# ============================================================
# 6. PREPROCESSOR
```

```
# ================================================================


def create_preprocessor(x):


    categorical_features = (

        x.select_dtypes(

            include=["object"]

        ).columns.tolist()

    )


    numerical_features = [

        column

        for column in x.columns

        if column not in categorical_features

    ]


    categorical_pipeline = Pipeline(

        steps=[

            (

                "imputer",

                SimpleImputer(

                    strategy="most_frequent"

                )

            ),
```

```
        (
            "encoder",
            OneHotEncoder(
                handle_unknown="ignore"
            )
        )
    ]
)

numerical_pipeline = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(
                strategy="median"
            )
        ),

        (
            "scaler",
            StandardScaler()
        )
    ]
```

```
    )

    preprocessor = ColumnTransformer(
        transformers=[
            (
                "categorical",
                categorical_pipeline,
                categorical_features
            ),

            (
                "numerical",
                numerical_pipeline,
                numerical_features
            )
        ]
    )

    return preprocessor

# ============================================================
# 7. MODEL COMPARISON
# ============================================================
```

```
def compare_models(
    x_train,
    x_test,
    y_train,
    y_test,
    preprocessor
):

    print("\n================================================")
    print("MODEL COMPARISON")
    print("================================================")

    models = {

        "Logistic Regression":
            LogisticRegression(
                max_iter=3000,
                class_weight="balanced",
                random_state=RANDOM_STATE
            ),

        "Random Forest":
            RandomForestClassifier(
                n_estimators=500,
```

```
            class_weight="balanced",
            random_state=RANDOM_STATE,
            n_jobs=-1
        ),

    "Extra Trees":
        ExtraTreesClassifier(
            n_estimators=500,
            class_weight="balanced",
            random_state=RANDOM_STATE,
            n_jobs=-1
        )
}

results = []

for name, model in models.items():

    pipeline = Pipeline(
        steps=[
            (
                "preprocessor",
                preprocessor
            ),
```

```
        (
            "model",
            model
        )
    ]
)

pipeline.fit(
    x_train,
    y_train
)

prediction = pipeline.predict(
    x_test
)

results.append({

    "Model": name,

    "Accuracy":
        accuracy_score(
            y_test,
```

```
        prediction
    ),

"Precision":
    precision_score(
        y_test,
        prediction,
        average="weighted",
        zero_division=0
    ),

"Recall":
    recall_score(
        y_test,
        prediction,
        average="weighted",
        zero_division=0
    ),

"F1":
    f1_score(
        y_test,
        prediction,
        average="weighted",
```

```
                zero_division=0
            )
        })

    results_df = pd.DataFrame(results)

    results_df = results_df.sort_values(
        by="F1",
        ascending=False
    )

    print(results_df.to_string(index=False))

    return results_df

# ============================================================
# 8. RANDOM FOREST HYPERPARAMETER TUNING
# ============================================================

def train_best_random_forest(
    x_train,
    y_train,
    preprocessor
):
```

```
print("\n================================================")
print("RANDOM FOREST HYPERPARAMETER TUNING")
print("================================================")

random_forest = RandomForestClassifier(
    random_state=RANDOM_STATE,
    class_weight="balanced",
    n_jobs=-1
)

pipeline = Pipeline(
    steps=[
        (
            "preprocessor",
            preprocessor
        ),

        (
            "model",
            random_forest
        )
    ]
)
```

```
parameter_grid = {

    "model__n_estimators":

        [300, 500, 700, 900],


    "model__max_depth":

        [None, 8, 12, 16, 20, 25],


    "model__min_samples_split":

        [2, 4, 6, 10],


    "model__min_samples_leaf":

        [1, 2, 3, 5],


    "model__max_features":

        ["sqrt", "log2", 0.5, 0.8]

}


cross_validator = StratifiedKFold(

    n_splits=5,

    shuffle=True,

    random_state=RANDOM_STATE

)
```

```
search = RandomizedSearchCV(
    estimator=pipeline,
    param_distributions=parameter_grid,
    n_iter=30,
    scoring="f1_weighted",
    cv=cross_validator,
    random_state=RANDOM_STATE,
    n_jobs=-1,
    verbose=1,
    refit=True
)

search.fit(
    x_train,
```

```
        y_train
    )

    print("\nBest Parameters:")

    for key, value in search.best_params_.items():
        print(key, "=", value)

    print(
        "\nBest Cross-Validation F1:",
        round(search.best_score_, 4)
    )

    return search.best_estimator_


# ============================================================
# 9. EVALUATE MODEL
# ============================================================

def evaluate_model(
    model,
    x_test,
    y_test
):
```

```
print("\n================================================")
print("FINAL MODEL EVALUATION")
print("================================================")

prediction = model.predict(
    x_test
)

accuracy = accuracy_score(
    y_test,
    prediction
)

precision = precision_score(
    y_test,
    prediction,
    average="weighted",
    zero_division=0
)

recall = recall_score(
    y_test,
    prediction,
```

```
    average="weighted",
    zero_division=0
)

f1 = f1_score(
    y_test,
    prediction,
    average="weighted",
    zero_division=0
)

print(
    f"\nAccuracy  : {accuracy:.4f}"
    f" ({accuracy * 100:.2f}%)"
)

print(
    f"Precision : {precision:.4f}"
    f" ({precision * 100:.2f}%)"
)

print(
    f"Recall    : {recall:.4f}"
    f" ({recall * 100:.2f}%)"
```

```
)

print(
    f"F1 Score  : {f1:.4f}"
    f" ({f1 * 100:.2f}%)"
)

print("\nClassification Report:\n")

print(
    classification_report(
        y_test,
        prediction,
        zero_division=0
    )
)

# ------------------------------------------------------------
# Confusion Matrix
# ------------------------------------------------------------

labels = sorted(
    y_test.unique()
)
```

```
cm = confusion_matrix(
    y_test,
    prediction,
    labels=labels
)

display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=labels
)

fig, ax = plt.subplots(
    figsize=(9, 7)
)

display.plot(
    ax=ax,
    xticks_rotation=30
)

plt.title(
    "BIS MarkGuard – Confusion Matrix"
)
```

```
    plt.tight_layout()

    plt.show()

    return prediction

# ============================================================
# 10. CROSS VALIDATION
# ============================================================

def cross_validation(
    model,
    x,
    y
):

    print("\n================================================")
    print("5-FOLD CROSS VALIDATION")
    print("================================================")

    cv = StratifiedKFold(
        n_splits=5,
        shuffle=True,
```

```
    random_state=RANDOM_STATE
)

scores = cross_val_score(
    model,
    x,
    y,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1
)

print(
    "\nFold accuracies:",
    np.round(scores, 4)
)

print(
    "\nMean CV Accuracy:",
    round(scores.mean(), 4)
)

print(
    "CV Standard Deviation:",
```

```
        round(scores.std(), 4)
    )

# ================================================================
# 11. FEATURE IMPORTANCE
# ================================================================

def show_feature_importance(model):

    print("\n================================================")
    print("FEATURE IMPORTANCE")
    print("================================================")

    preprocessor = (
        model.named_steps["preprocessor"]
    )

    random_forest = (
        model.named_steps["model"]
    )

    feature_names = (
        preprocessor
        .get_feature_names_out()
```

```
)

importance = (
    random_forest
    .feature_importances_
)

importance_df = pd.DataFrame({

    "Feature":
        feature_names,

    "Importance":
        importance
})

importance_df = (
    importance_df
    .sort_values(
        by="Importance",
        ascending=False
    )
    .head(25)
)
```

```
print(
    importance_df
    .to_string(index=False)
)

plt.figure(
    figsize=(10, 8)
)

plt.barh(
    importance_df["Feature"].iloc[::-1],
    importance_df["Importance"].iloc[::-1]
)

plt.title(
    "Top 25 BIS MarkGuard Features"
)

plt.xlabel(
    "Feature Importance"
)

plt.tight_layout()
```

```
    plt.show()


# ================================================================
# 12. SAVE MODEL
# ================================================================


def save_model(model, x):


    package = {


        "model": model,


        "feature_columns":

            x.columns.tolist(),


        "description":

            "BIS MarkGuard Random Forest Risk Prediction Model"

    }


    joblib.dump(

        package,

        MODEL_FILE

    )
```

```
    print(
        "\nModel saved successfully as:"
    )

    print(
        MODEL_FILE
    )

# ============================================================
# 13. PREDICT NEW PRODUCT
# ============================================================

def predict_new_product(
    model,
    training_columns
):

    print("\n================================================")
    print("BIS PRODUCT PREDICTION")
    print("================================================")

    print(
        "\nEnter the product information below."
```

```
)

product_name = input(
    "\nProduct Name: "
)

product_category = input(
    "Product Category: "
)

observed_claim = input(
    "Observed BIS Claim: "
)

license_number = input(
    "BIS Licence Number: "
)

is_number = input(
    "IS Number: "
)

manufacturer = input(
    "Manufacturer / Licence Holder: "
```

```
)

new_product = pd.DataFrame([{

    "product_name":

        product_name,

    "product_category":

        product_category,

    "observed_claim":

        observed_claim,

    "candidate_license_number":

        license_number,

    "candidate_is_number":

        is_number,

    "claimed_license_holder":

        manufacturer

}])

# Feature engineering
```

```
new_product = build_features(
    new_product
)

# Ensure exact training columns
for column in training_columns:

    if column not in new_product.columns:

        new_product[column] = np.nan

new_product = new_product[
    training_columns
]

# ------------------------------------------------------------
# Prediction
# ------------------------------------------------------------

prediction = model.predict(
    new_product
)[0]

probabilities = (
```

```
    model.predict_proba(
        new_product
    )[0]
)

classes = model.classes_

probability_df = pd.DataFrame({

    "Risk Class":
        classes,

    "Probability":
        probabilities
})

probability_df = (
    probability_df
    .sort_values(
        by="Probability",
        ascending=False
    )
    .reset_index(drop=True)
)
```

```
confidence = (
    probability_df
    .iloc[0]["Probability"]
)

# ------------------------------------------------------------
# Display result
# ------------------------------------------------------------

print("\n================================================")
print("PREDICTION RESULT")
print("================================================")

print(
    "\nPredicted Risk Class:",
    prediction
)

print(
    f"Model Confidence: "
    f"{confidence * 100:.2f}%"
)
```

```
print("\nClass Probabilities:")


print(
    probability_df
    .to_string(index=False)
)


# ------------------------------------------------------------
# Rule-based warnings
# ------------------------------------------------------------

warnings_list = []

if not license_number.strip():

    warnings_list.append(
        "No BIS licence number was provided."
    )

if (
    "DEMO"
    in license_number.upper()
):
```

```
        warnings_list.append(
            "Licence number contains DEMO/test-data text."
        )

    if not re.search(
        r"\d",
        license_number
    ):

        warnings_list.append(
            "Licence number contains no numeric characters."
        )

    if (
        is_number.strip()
        and not re.match(
            r"^IS\s+\d+",
            is_number.strip(),
            re.IGNORECASE
        )
    ):

        warnings_list.append(
            "IS number format could not be recognized."
```

```
    )


if (
    not observed_claim.strip()
    or
    "no clear"
    in observed_claim.lower()
):

    warnings_list.append(
        "Certification claim is missing or unclear."
    )

print(
    "\n================================================"
)

print(
    "MARKGUARD WARNING CHECK"
)

print(
    "================================================"
)
```

```
if warnings_list:

    for warning in warnings_list:

        print(
            "WARNING:",
            warning
        )

else:

    print(
        "No basic warning flags detected."
    )

print(
    "\nIMPORTANT:"
)

print(
    "This prediction is a risk assessment."
)
```

```
    print(
        "It is NOT by itself proof that a product is"
        " genuine or counterfeit."
    )

# ============================================================
# 14. MAIN PROGRAM
# ============================================================

def main():

    print("\n")
    print("============================================================")
    print("              BIS MARKGUARD AI SYSTEM")
    print("       BIS PRODUCT AUTHENTICITY RISK PREDICTION")
    print("============================================================")

    # ------------------------------------------------------------
    # Load dataset
    # ------------------------------------------------------------

    data = load_dataset()

    if data is None:
```

```
    return


# ------------------------------------------------------------
# Dataset information
# ------------------------------------------------------------

dataset_information(
    data
)

# ------------------------------------------------------------
# Prepare data
# ------------------------------------------------------------

print(
    "\nPreparing features..."
)

x, y = prepare_training_data(
    data
)

print(
```

```
    "\nFinal number of features:",
    x.shape[1]
)

print(
    "Target classes:",
    sorted(y.unique())
)

# ------------------------------------------------------------
# Train/test split
# ------------------------------------------------------------

x_train, x_test, y_train, y_test = (
    train_test_split(
        x,
        y,
        test_size=0.20,
        random_state=RANDOM_STATE,
        stratify=y
    )
)

print(
```

```
    "\nTraining records:",
    len(x_train)
)

print(
    "Testing records:",
    len(x_test)
)

# ------------------------------------------------------------
# Preprocessor
# ------------------------------------------------------------

preprocessor = create_preprocessor(
    x
)

# ------------------------------------------------------------
# Compare models
# ------------------------------------------------------------

compare_models(
    x_train,
    x_test,
```

```
    y_train,
    y_test,
    preprocessor
)

# ------------------------------------------------------------
# Train best Random Forest
# ------------------------------------------------------------

best_model = train_best_random_forest(
    x_train,
    y_train,
    preprocessor
)

# ------------------------------------------------------------
# Evaluate
# ------------------------------------------------------------

evaluate_model(
    best_model,
    x_test,
    y_test
)
```

```
# ------------------------------------------------------------
# Cross-validation
# ------------------------------------------------------------

cross_validation(
    best_model,
    x,
    y
)

# ------------------------------------------------------------
# Feature importance
# ------------------------------------------------------------

show_feature_importance(
    best_model
)

# ------------------------------------------------------------
# Save model
# ------------------------------------------------------------

save_model(
```

```
    best_model,
    x
)

# ------------------------------------------------------------
# Prediction menu
# ------------------------------------------------------------

while True:

    print("\n")
    print("================================================")
    print("MARKGUARD MENU")
    print("================================================")

    print(
        "1. Predict a new BIS product"
    )

    print(
        "2. Exit"
    )

    choice = input(
```

```
    "\nEnter choice: "
)

if choice == "1":

    predict_new_product(
        best_model,
        x.columns.tolist()
    )

elif choice == "2":

    print(
        "\nThank you for using BIS MarkGuard."
    )

    break

else:

    print(
        "\nInvalid choice."
    )
```

```
# ================================================================

# PROGRAM START

# ================================================================


if __name__ == "__main__":


    main()
```